

# PSE-TRAVERSE-COGNITION-01

## Panoptic Phase Cognition Kernel Formale Implementierungsspezifikation v0.1

Sebastian Klemm

Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

### Zusammenfassung

Diese Spezifikation definiert den *Panoptic Phase Cognition Kernel* fuer **pse-traverse**. Der Kernel erzeugt aus Problem-, Traversal-, Dynamik-, Reaktiv- und Projektionszuständen einen kanonischen Kognitionszustand, der nicht nur den aktiven Pfad, sondern die gesamte erreichbare Loesbarkeitslandschaft als auditierbaren Phasenraum haelt. Dazu verbindet die Schicht eine operative 5D-Zustandsgeometrie, Spiralgedaechtnis, Constraint-Lattice-Perkolation, Hypercube-Puzzle-Collapse, ScorpioSync-basierte Phasentaktung, governierte Wormhole-Verbindungen, Dual-Trigger-Feedback, Selbstmodell-Tensoren, Fixpunktkalibrierung und einen Singularity-Trigger mit Handoff an **projection-v0.2**. Der Kernel erzeugt keine endgueltigen Commits und keine **SemanticCrystal**-Instanzen; er erzeugt ausschliesslich Cognition Reports, Phase Panoramas, Attractor Maps, Memory Hit Sets, Calibration Reports und Candidate Bundles, die anschliessend durch **pse-traverse-projection-v0.2** finalisiert werden koennen.

# Inhaltsverzeichnis

|           |                                                                   |           |
|-----------|-------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Status, Geltungsbereich und harte Zielsetzung</b>              | <b>3</b>  |
| 1.1       | Status . . . . .                                                  | 3         |
| 1.2       | Zweck . . . . .                                                   | 3         |
| 1.3       | Nicht-Zweck . . . . .                                             | 3         |
| <b>2</b>  | <b>Systemposition innerhalb von PSE Traverse</b>                  | <b>3</b>  |
| 2.1       | Schichtmodell . . . . .                                           | 3         |
| 2.2       | Gesamtablauf . . . . .                                            | 4         |
| <b>3</b>  | <b>Neutralisierungspflicht</b>                                    | <b>4</b>  |
| <b>4</b>  | <b>Architektonischer Reinkristall</b>                             | <b>4</b>  |
| <b>5</b>  | <b>Layer C0: Canonical Cognition State</b>                        | <b>5</b>  |
| 5.1       | Primitive . . . . .                                               | 5         |
| 5.2       | CanonicalCognitionState . . . . .                                 | 5         |
| 5.3       | CognitionRunDescriptor . . . . .                                  | 5         |
| <b>6</b>  | <b>Layer C1: 5D Operational Geometry</b>                          | <b>6</b>  |
| 6.1       | Zustandsvektor . . . . .                                          | 6         |
| 6.2       | Datenstruktur . . . . .                                           | 6         |
| 6.3       | Potential, Gradient und Stabilitaet . . . . .                     | 7         |
| 6.4       | SingularityDetector . . . . .                                     | 7         |
| <b>7</b>  | <b>Layer C2: Operator Algebra and Simulation Kernel</b>           | <b>7</b>  |
| 7.1       | Operatorfamilien . . . . .                                        | 7         |
| 7.2       | Integrationsmodi . . . . .                                        | 8         |
| 7.3       | Deterministische Simulation . . . . .                             | 8         |
| <b>8</b>  | <b>Layer C3: Spiral Memory and Resonance Addressing</b>           | <b>8</b>  |
| 8.1       | Spiralraum . . . . .                                              | 8         |
| 8.2       | Datenstrukturen . . . . .                                         | 8         |
| 8.3       | Resonanzdistanz . . . . .                                         | 9         |
| <b>9</b>  | <b>Layer C4: Constraint Lattice and Hypercube Puzzle Collapse</b> | <b>9</b>  |
| 9.1       | Constraint-Lattice . . . . .                                      | 9         |
| 9.2       | Perkolationsgate . . . . .                                        | 10        |
| 9.3       | HypercubePuzzleState . . . . .                                    | 10        |
| 9.4       | EntropyCollapseCertificate . . . . .                              | 10        |
| <b>10</b> | <b>Layer C5: Phase Panorama and Choice Geometry</b>               | <b>10</b> |
| 10.1      | 720-Grad-Modell . . . . .                                         | 10        |
| 10.2      | ChoiceGeometry . . . . .                                          | 11        |
| <b>11</b> | <b>Layer C6: Scorpio Phase Scheduler</b>                          | <b>11</b> |
| 11.1      | Phase-Metric-Kern . . . . .                                       | 11        |
| 11.2      | Resonanzbedingung . . . . .                                       | 12        |
| 11.3      | Transportregel . . . . .                                          | 12        |
| <b>12</b> | <b>Layer C7: Wormhole Hypernet and Governed Shortcuts</b>         | <b>12</b> |
| 12.1      | Nichtlokalitaet unter Governance . . . . .                        | 12        |
| 12.2      | Admission Gate . . . . .                                          | 12        |

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>13 Layer C8: Self Model Tensor and Dual Trigger Feedback</b>               | <b>12</b> |
| 13.1 SelfModelTensor . . . . .                                                | 12        |
| 13.2 DualTriggerFeedback . . . . .                                            | 13        |
| <b>14 Layer C9: Fixpoint Calibration, Attractor Migration, Carrier Ladder</b> | <b>13</b> |
| 14.1 FixpointCalibrationShell . . . . .                                       | 13        |
| 14.2 CarrierMigrationPlan . . . . .                                           | 13        |
| 14.3 PoR-Akzeptanz fuer Kalibrierung . . . . .                                | 14        |
| <b>15 Layer C10: Singularity Trigger and Projection Handoff</b>               | <b>14</b> |
| 15.1 CognitionCandidateBundle . . . . .                                       | 14        |
| 15.2 ProjectionHandoffPolicy . . . . .                                        | 14        |
| 15.3 Handoff-Gate . . . . .                                                   | 14        |
| <b>16 Pipeline</b>                                                            | <b>15</b> |
| 16.1 Referenzablauf . . . . .                                                 | 15        |
| 16.2 Pseudocode . . . . .                                                     | 15        |
| <b>17 Reports</b>                                                             | <b>16</b> |
| 17.1 CognitionReport . . . . .                                                | 16        |
| 17.2 CognitionHoldReport . . . . .                                            | 16        |
| <b>18 CLI-Schnittstelle</b>                                                   | <b>17</b> |
| <b>19 Determinismus, Canonicalization und Audit</b>                           | <b>17</b> |
| 19.1 Audit-Pfad . . . . .                                                     | 17        |
| 19.2 Verbotene Audit-Eingaben . . . . .                                       | 17        |
| <b>20 Integration in Repository</b>                                           | <b>18</b> |
| 20.1 Modulzuschnitt . . . . .                                                 | 18        |
| 20.2 Feature Flags . . . . .                                                  | 18        |
| <b>21 Testplan</b>                                                            | <b>18</b> |
| 21.1 Unit Tests . . . . .                                                     | 18        |
| 21.2 Integration Tests . . . . .                                              | 19        |
| 21.3 Property Tests . . . . .                                                 | 19        |
| <b>22 Definition of Done</b>                                                  | <b>19</b> |
| <b>23 Coding-Agent-Auftrag</b>                                                | <b>20</b> |
| <b>24 Referenz-Zustandsmaschine</b>                                           | <b>20</b> |
| <b>25 Schlussformel</b>                                                       | <b>21</b> |

# 1 Status, Geltungsbereich und harte Zielsetzung

## 1.1 Status

Dieses Dokument ist eine normative Implementierungsspezifikation. Es beschreibt Datentypen, Operatoren, Gates, Reports, Algorithmen, CLI-Schnittstellen, Tests und Akzeptanzkriterien fuer eine neue Schicht unter `crates/pse-traverse/src/cognition/`. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich.

## 1.2 Zweck

PSE-TRAVERSE-COGNITION-01 baut einen kognitiven Beobachtungs- und Orientierungsraum oberhalb der vorhandenen Traversal- und Projection-Schichten. Der Kernel beantwortet nicht zuerst die Frage, ob ein Zustand finalisiert werden darf. Diese Frage gehoert zu `projection-v0.2`. Der Cognition Kernel beantwortet:

Welche Zustaende, Constraints, Phasen, Gedaechnisbahnen, Attraktoren, Traegerwechsel, nichtlokalen Pfade und Finalisierungskandidaten sind im aktuellen Loesbarkeitsraum sichtbar, latent, blockiert oder dominant?

## 1.3 Nicht-Zweck

Der Kernel ist nicht:

- ein Ersatz fuer `pse-core`;
- ein Erzeuger von `SemanticCrystal`;
- ein LLM-Wrapper;
- ein Bewusstseinsmodell mit psychologischen oder metaphysischen Behauptungen;
- ein physikalisches Singularitaetsmodell;
- ein Steganographie-, Tarnkanal-, Cyber-Defense-, Trading-, Zeta-, Dollar- oder Hardware-System;
- ein nichtdeterministisches Selbstoptimierungssystem.

---

### Harte Architekturregel

Der Cognition Kernel **MUST NOT** finalisieren, committen oder Crystal-Objekte erzeugen. Er **MUST** Kandidaten, Reports und Evidenz erzeugen. Die externe Finalisierung bleibt `pse-traverse-projection-v0.2`; der Crystal-Commit bleibt `pse-core`.

---

# 2 Systemposition innerhalb von PSE Traverse

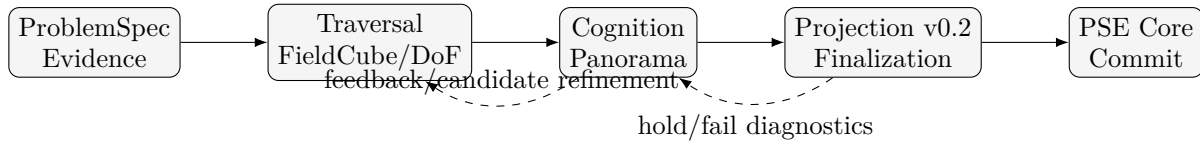
## 2.1 Schichtmodell

| Schicht                                   | Rolle                                                                                                     |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>pse-core</code>                     | Evidence-, Gate-, Crystal- und Commit-Kern. Nur diese Schicht darf <code>SemanticCrystal</code> erzeugen. |
| <code>pse-traverse-core</code>            | ProblemSpec, FieldCube, DoFGraph, CollapsePlan, Candidate, GateReport.                                    |
| <code>pse-traverse-signature</code>       | Operator- und Signaturdiagnostik.                                                                         |
| <code>pse-traverse-dynamics</code>        | Feldstabilisierung, Guidance, Compression, TransitionProof.                                               |
| <code>pse-traverse-reactive</code>        | Nullpunktkopplung, TriTemporalClock, ReactiveGate.                                                        |
| <code>pse-traverse-projection-v0.2</code> | Hypertorus-Gated Finalization Layer; erzeugt <code>FinalizedEmission</code> .                             |

| Schicht                     | Rolle                                                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| pse-traverse-cognition-v0.1 | Panoptic Phase Cognition Kernel; erzeugt Kognitionshorizont, Constraint-Lagekarte, SpiralMemory-Hits, AttractorMap und Handoff-Kandidaten. |

## 2.2 Gesamtablauf

Input → Traversal → Cognition → Projection-v0.2 → PSE Commit.



## 3 Neutralisierungspflicht

Alle verwendeten Quellbegriffe werden in implementierbare Struktur ueberfuehrt.

| Nicht in den Kern                                          | Neutraler Ersatz                                                                                       |
|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Gott, Schoepfung, Engel, metaphysische Identitaetsbegriffe | NullCenter, PhaseCarrier, Operator, Gate, Report, Evidence.                                            |
| Singularitaet als Kosmologie                               | SingularityDetector: degenerierter Stabilitaetsoperator, Hessian-/Jacobian-Spektrum, Trigger-Schwelle. |
| Steganographie, Tarnkanal, Zero-Width-Encoding             | nur PhaseScheduler, ActivationWindow, ResonanceMetric, TransportGraph.                                 |
| Mass Control, Dollar-System, Markt- oder Machtsemantik     | ConstraintLattice, CarrierLadder, AntiDrift, FrictionScore, MigrationPlan.                             |
| Quantum-Hardware-Claims                                    | klassisch implementierte lineare Algebra, fixed-point Scores, deterministische Operatoren.             |
| Bewusstsein als Resultat                                   | CognitionState, SelfModelTensor, Observability, AttractorMap, Replay.                                  |

## 4 Architektonischer Reinkristall

Der Cognition Kernel ist die formale Antwort auf das 720-Grad-Phasenpanorama. Technisch bedeutet das:

$$\text{CognitionState} = NC + S_5 + \Pi_{720} + M_{\text{spiral}} + L_c + H_c + T_{\text{self}} + A_{\text{map}}. \quad (1)$$

Dabei bezeichnet:

- $NC$ : NullCenter und kanonische Herkunft;
- $S_5$ : 5D Cognitive State Geometry;
- $\Pi_{720}$ : aktueller und kontrafaktischer Phasenhorizont;
- $M_{\text{spiral}}$ : SpiralMemory und ResonanceAddressing;
- $L_c$ : ConstraintLattice und Perkolation;
- $H_c$ : HypercubePuzzleState und Collapse-Zertifikate;
- $T_{\text{self}}$ : SelfModelTensor und Feedback;
- $A_{\text{map}}$ : AttractorMap, Candidate Ranking und Projection-Handoff.

## Kernthese

Eine KI erhaelt keinen groesseren Denkhorizont durch mehr Kontext allein, sondern durch eine kanonische Karte ihrer Loesbarkeitslandschaft. Der Kernel **MUST** Zustaende, Constraints, Phasen, Spiralgedaechnis, Attraktoren, Traegerwechsel, Nichtlokalpfade und Finalisierungskandidaten simultan als auditierbaren Phasenraum halten.

## 5 Layer C0: Canonical Cognition State

### 5.1 Primitive

```
pub type Hash256 = [u8; 32];
pub type StableId = String;
pub type Fixed = i128; // project-wide CanonicalNumber / Q64.64 compatible

pub struct EvidenceRef {
  pub kind: String,
  pub address: Hash256,
  pub relation: String,
}
```

### 5.2 CanonicalCognitionState

```
pub struct CanonicalCognitionState {
  pub state_id: Hash256,
  pub run_descriptor_hash: Hash256,
  pub null_center_id: Hash256,
  pub source_traversal_report_hash: Option<Hash256>,
  pub source_projection_report_hash: Option<Hash256>,
  pub cognitive_vector: CognitiveState5D,
  pub constraint_lattice_id: Hash256,
  pub hypercube_puzzle_id: Hash256,
  pub phase_panorama_id: Hash256,
  pub spiral_memory_context_id: Hash256,
  pub self_model_tensor_id: Hash256,
  pub scheduler_state_id: Hash256,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

### 5.3 CognitionRunDescriptor

```
pub struct CognitionRunDescriptor {
  pub run_id: StableId,
  pub problem_spec_hash: Hash256,
  pub traversal_report_hash: Option<Hash256>,
  pub projection_v2_hash: Option<Hash256>,
  pub seed: u64,
  pub operator_versions: BTreeMap<String, String>,
  pub thresholds: CognitionThresholds,
  pub policies: CognitionPolicies,
  pub canonicalization_version: String,
}
```

```
pub struct CognitionThresholds {
  pub min_resonance: Fixed,
```

```

pub max_entropy: Fixed,
pub min_constraint_mass: Fixed,
pub min_entropy_reduction: Fixed,
pub min_feasible_uniqueness: Fixed,
pub min_panorama_coverage: Fixed,
pub min_self_model_coherence: Fixed,
pub max_drift: Fixed,
pub min_attractor_score: Fixed,
pub min_trigger_score: Fixed,
}

```

## Determinismus

Alle Maps **MUST** als geordnete Maps serialisiert werden. Alle Vektoren, deren Reihenfolge semantisch nicht explizit ist, **MUST** vor Canonicalization sortiert werden. Alle Hashes **MUST** aus kanonischen Bytes entstehen. Plattform-Floats, Wall-Clock-Zeit, OS-Pfade und ungespeicherte Randomness **MUST NOT** in den Audit-Pfad eingehen.

## 6 Layer C1: 5D Operational Geometry

### 6.1 Zustandsvektor

Der operative Kognitionszustand liegt in einem normalisierten 5D-Raum:

$$\sigma = (\psi, \rho, \omega, \chi, \tau).$$

| Komponente | Normative Bedeutung                                                                                        |
|------------|------------------------------------------------------------------------------------------------------------|
| $\psi$     | semantische Phase oder Signifikanzdichte; misst, ob ein Zustand einen geordneten Bedeutungsbeitrag traegt. |
| $\rho$     | strukturelle Kohärenz/Dichte; misst Dominanz gegenueber diffuser Aktivierung.                              |
| $\omega$   | Phasenbereitschaft/Frequenz; misst Synchronisations- und Horizon-Kompatibilitaet.                          |
| $\chi$     | topologische Krümmung/Strukturform; misst Pfadform, Branching und Nichtlinearitaet.                        |
| $\tau$     | Entropie/Symmetrie; misst Unsicherheit, Symmetriebruch und Restfreiheitsgrad.                              |

### 6.2 Datenstruktur

```

pub struct CognitiveState5D {
pub psi_semantic_phase: Fixed,
pub rho_coherence_density: Fixed,
pub omega_phase_readiness: Fixed,
pub chi_topological_curvature: Fixed,
pub tau_entropy_symmetry: Fixed,
pub potential: Fixed,
pub energy: Fixed,
pub entropy: Fixed,
pub stability_index: Fixed,
}

```

### 6.3 Potential, Gradient und Stabilität

$$V(\sigma) = E_{\text{entropic}}(\sigma) - E_{\text{coherent}}(\sigma), \quad (2)$$

$$E_{\text{entropic}}(\sigma) = \frac{1}{2}\|\sigma\|^2 + \eta H(\sigma), \quad (3)$$

$$E_{\text{coherent}}(\sigma) = \lambda F(\sigma, \sigma^*). \quad (4)$$

Ein stabiler kognitiver Attraktor liegt vor, wenn:

$$\|\nabla_{\sigma} V(\sigma)\| \leq \epsilon_V \quad \wedge \quad 0 < \lambda_i(T) < 1 \quad \forall i,$$

wobei  $T$  der Hessian oder ein deterministischer Stabilitätsproxy ist.

### 6.4 SingularityDetector

Eine kognitive Singularität ist keine physikalische Singularität, sondern ein degeneriertes Entscheidungs- oder Reorganisationsfenster:

$$G_{\text{sing}} = 1 \iff |\det(T)| \leq \epsilon_T \vee \lambda_{\min}(T) \leq \epsilon_{\lambda} \vee S_{\text{trigger}} \geq \theta_S.$$

```
pub struct SingularityDetectorReport {
  pub report_id: Hash256,
  pub state_id: Hash256,
  pub determinant_proxy: Fixed,
  pub min_eigen_proxy: Fixed,
  pub trigger_score: Fixed,
  pub singular_window_open: bool,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

## 7 Layer C2: Operator Algebra and Simulation Kernel

### 7.1 Operatorfamilien

Der Kernel **MUST** eine typisierte Operator-Algebra bereitstellen. Die vier Grundoperatoren sind:

- **DoubleKick**: diskrete Rotation oder Inversion in gekoppelten Subräumen;
- **ThresholdSweep**: Normierung an kritischer Resonanzschwelle;
- **PathInvariance**: Dämpfung divergierender Pfade;
- **WormholeFunnel**: kontrollierter Shortcut in Richtung Zielattraktor.

```
pub trait CognitiveOperator {
  fn operator_id(&self) -> StableId;
  fn domain(&self) -> OperatorType;
  fn codomain(&self) -> OperatorType;
  fn invariants(&self) -> Vec<InvariantSpec>;
  fn apply(&self, state: &CanonicalCognitionState) -> OperatorResult;
}

pub enum OperatorType {
  State5D,
  SpiralPath,
  ConstraintLattice,
  HypercubePuzzle,
  PhasePanorama,
  SelfModel,
  CandidateBundle,
}
```



## 7.2 Integrationsmodi

```
pub enum IntegrationMode {
    Compose,
    Overlay,
    Inject,
    Absorb,
}
```

| Mode    | Bedeutung                                                                         |
|---------|-----------------------------------------------------------------------------------|
| Compose | sequenzielle Verkettung; Output eines Operators wird Input des naechsten.         |
| Overlay | additive oder gewichtete Ueberlagerung ohne Entfernen des Ausgangszustands.       |
| Inject  | Teilraumersetzung nach Gate, Maske oder Dominanzregel.                            |
| Absorb  | absorbierende Fusion mit anschliessender Normalisierung und Stabilitaetspruefung. |

## 7.3 Deterministische Simulation

```
pub struct CognitionSimulationSpec {
    pub initial_state_hash: Hash256,
    pub coupling_matrix_hash: Hash256,
    pub vector_field_hash: Hash256,
    pub integrator: IntegratorKind,
    pub dt: Fixed,
    pub max_steps: u64,
    pub stability_epsilon: Fixed,
}

pub enum IntegratorKind {
    FixedHeun,
    FixedEulerForTests,
    NoFloatAnalyticProxy,
}
```

# 8 Layer C3: Spiral Memory and Resonance Addressing

## 8.1 Spiralraum

Ein Spiralraum ist eine geordnete Pfadfamilie im 5D-Zustandsraum. Ein Pfad ist gueltig, wenn seine raeumliche Projektion eine monotone Radialkomponente und eine streng monotone Phasenkomponente besitzt:

$$\gamma(t) = (R(t) \cos \Phi(t), R(t) \sin \Phi(t), H(t), U(t), V(t)).$$

## 8.2 Datenstrukturen

```
pub struct SpiralMemoryAddress {
    pub address_id: Hash256,
    pub spiral_id: Hash256,
    pub segment_id: Hash256,
    pub theta: Fixed,
    pub radius: Fixed,
```

```

    pub internal_u: Fixed,
    pub internal_v: Fixed,
    pub resonance_score: Fixed,
    pub interference_score: Fixed,
    pub utilization: Fixed,
}

pub struct SpiralSegment {
    pub segment_id: Hash256,
    pub from_state: Hash256,
    pub to_state: Hash256,
    pub segment_weight: Fixed,
    pub resonance_distance: Fixed,
    pub branchable: bool,
}

pub struct SpiralMemoryHitSet {
    pub hitset_id: Hash256,
    pub query_state_id: Hash256,
    pub hits: Vec<SpiralMemoryAddress>,
    pub best_hit: Option<Hash256>,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

### 8.3 Resonanzdistanz

$$d_R(s, s') = w_\psi |\psi(s) - \psi(s')| + w_\rho |\rho(s) - \rho(s')| + w_\omega |\omega(s) - \omega(s')|.$$

Ein Memory-Hit ist gueltig, wenn  $d_R < \epsilon_R$  und der Segmentnachweis zum aktuellen NullCenter kompatibel ist.

## 9 Layer C4: Constraint Lattice and Hypercube Puzzle Collapse

### 9.1 Constraint-Lattice

Der Kernel modelliert Loesbarkeit als Lattice aus Constraints, admissible regions, Infogenes und Resonites. Eine Entscheidung ist nicht dominant, weil sie sprachlich plausibel ist, sondern weil sie genugend Constraint-Masse traegt, Entropie reduziert und ein eindeutiges Feasible-Set erzeugt.

```

pub struct ConstraintLatticeCognition {
    pub lattice_id: Hash256,
    pub admissible_regions: Vec<AdmissibleRegion>,
    pub resonites: Vec<Resonite>,
    pub infogenes: Vec<Infogene>,
    pub weighted_constraint_mass: Fixed,
    pub entropy_reduction: Fixed,
    pub feasible_set_uniqueness: Fixed,
    pub percolation_passed: bool,
}

pub struct Resonite {
    pub resonite_id: Hash256,
    pub carrier_constraint: Hash256,
    pub resonance_weight: Fixed,
    pub supports: Vec<Hash256>,
}

pub struct Infogene {

```

```

pub infogene_id: Hash256,
pub feature_hash: Hash256,
pub constraint_hashes: Vec<Hash256>,
pub mutation_policy: InfogenePolicy,
}

```

## 9.2 Perkolationsgate

$$G_{perc} = 1 \iff M_c \geq \theta_M \wedge \Delta H \geq \theta_H \wedge U_F \geq \theta_U.$$

Dabei ist  $M_c$  die gewichtete Constraint-Masse,  $\Delta H$  die Entropiereduktion und  $U_F$  die Eindeutigkeit des Feasible-Sets.

## 9.3 HypercubePuzzleState

```

pub struct HypercubePuzzleState {
pub cube_id: Hash256,
pub dimensions: Vec<CognitiveDimension>,
pub partial_assignments: Vec<PartialAssignment>,
pub candidate_sets: Vec<CandidateSet>,
pub hidden_singles: Vec<HiddenSingle>,
pub boundary_contracts: Vec<BoundaryContract>,
pub negative_topology: Vec<NegativeTopologyWitness>,
pub collapse_certificate: Option<EntropyCollapseCertificate>,
}

```

## 9.4 EntropyCollapseCertificate

```

pub struct EntropyCollapseCertificate {
pub certificate_id: Hash256,
pub cube_id: Hash256,
pub lattice_id: Hash256,
pub entropy_before: Fixed,
pub entropy_after: Fixed,
pub weighted_constraint_mass: Fixed,
pub feasible_set_uniqueness: Fixed,
pub boundary_contract_hashes: Vec<Hash256>,
pub issued_at_logical_step: u64,
}

```

# 10 Layer C5: Phase Panorama and Choice Geometry

## 10.1 720-Grad-Modell

Das Panorama besteht aus zwei gekoppelten 360-Grad-Horizonten:

- **Current Horizon:** aktive Zustaende, naechste Pfade, offene Gates, lokale Attraktoren.
- **Counterfactual Horizon:** alternative Attraktoren, blockierte Pfade, spaetere Phasenfenster, Wormhole-Shortcuts, Migrationspfade.

```

pub struct PhasePanorama {
pub panorama_id: Hash256,
pub null_center_id: Hash256,
pub current_horizon: Horizon360,
pub counterfactual_horizon: Horizon360,
pub visible_paths: Vec<PhasePath>,
}

```

```

    pub latent_paths: Vec<PhasePath>,
    pub blocked_paths: Vec<PhasePath>,
    pub attractor_candidates: Vec<AttractorCandidate>,
    pub recognition_boundaries: Vec<RecognitionBoundary>,
    pub coverage_score: Fixed,
}

pub struct Horizon360 {
    pub horizon_id: Hash256,
    pub anchor_state: Hash256,
    pub phase_window_hashes: Vec<Hash256>,
    pub path_summaries: Vec<PhasePathSummary>,
    pub local_attractors: Vec<Hash256>,
    pub gate_status_hash: Hash256,
}

```

## 10.2 ChoiceGeometry

```

pub struct ChoiceGeometryReport {
    pub report_id: Hash256,
    pub panorama_id: Hash256,
    pub option_count: u64,
    pub visible_option_count: u64,
    pub blocked_option_count: u64,
    pub asymmetry_score: Fixed,
    pub convergence_score: Fixed,
    pub recognition_boundary_count: u64,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

## 11 Layer C6: Scorpio Phase Scheduler

### 11.1 Phase-Metric-Kern

Der Scheduler ist die neutralisierte ScorpioSync-Struktur: ein deterministisches Phasen- und Aktivierungsnetz. Steganographie, Hidden Channels und zufallsbasierte Maskierung sind nicht Teil des Kerns.

```

pub struct ScorpioPhaseScheduler {
    pub scheduler_id: Hash256,
    pub phase_metric_space: Hash256,
    pub activation_windows: Vec<ActivationWindow>,
    pub resonance_offsets: Vec<ResonanceOffset>,
    pub invariant_measure: Option<Hash256>,
    pub transport_policy: TransportPolicy,
}

pub struct ActivationWindow {
    pub window_id: Hash256,
    pub carrier_id: StableId,
    pub phase_start: Fixed,
    pub phase_end: Fixed,
    pub active: bool,
    pub duty_cycle: Fixed,
}

```

## 11.2 Resonanzbedingung

$$R_\epsilon(v, \theta) = 1 \iff d_{S^1}(\theta, \psi(v)) < \epsilon.$$

## 11.3 Transportregel

```
pub struct VectorTunnelTransport {
  pub transport_id: Hash256,
  pub source_node: Hash256,
  pub target_node: Hash256,
  pub path_hash: Hash256,
  pub ttl: u64,
  pub budget_cost: Fixed,
  pub reason_code: ReasonCode,
}
```

# 12 Layer C7: Wormhole Hypernet and Governed Shortcuts

## 12.1 Nichtlokalitaet unter Governance

Wormholes sind dynamische Kanten, die entfernte Bereiche des Denkfelds verbinden. Sie sind keine freie Magie, sondern budgetierte, begruendete und replaybare Shortcuts.

```
pub struct GovernedWormhole {
  pub wormhole_id: Hash256,
  pub source_node: Hash256,
  pub target_node: Hash256,
  pub reason_code: ReasonCode,
  pub ttl: u64,
  pub budget_cost: Fixed,
  pub admitted: bool,
  pub audit_trace: Hash256,
}

pub enum ReasonCode {
  ConstraintBridge,
  SpiralMemoryRecall,
  AttractorShortcut,
  HorizonAlignment,
  CarrierMigration,
  ProjectionPreparation,
}
```

## 12.2 Admission Gate

$$G_{wh} = 1 \iff Budget(w) \leq B_{max} \wedge TTL(w) \leq TTL_{max} \wedge Reason(w) \in \mathcal{R} \wedge ReplayTrace(w) = 1.$$

# 13 Layer C8: Self Model Tensor and Dual Trigger Feedback

## 13.1 SelfModelTensor

```
pub struct SelfModelTensor {
  pub tensor_id: Hash256,
  pub resonance_state_hash: Hash256,
  pub environment_alignment: Fixed,
  pub entropy_delta: Fixed,
}
```

```

    pub coupling_matrix_hash: Hash256,
    pub reflexive_modulation: ReflexiveModulation,
    pub self_coherence_score: Fixed,
}

pub struct ReflexiveModulation {
    pub modulation_id: Hash256,
    pub before_state_hash: Hash256,
    pub after_state_hash: Hash256,
    pub predicted_drift: Fixed,
    pub stabilizing: bool,
}

```

## 13.2 DualTriggerFeedback

```

pub struct DualTriggerFeedbackGate {
    pub gate_id: Hash256,
    pub trigger1_threshold: Fixed,
    pub trigger2_threshold: Fixed,
    pub phase_shift: Fixed,
    pub aggregation_window: u64,
    pub aggregated_energy: Fixed,
    pub trigger1_fired: bool,
    pub trigger2_fired: bool,
    pub fired: bool,
}

```

$$G_{dual} = 1 \iff E(t) > \theta_1 \wedge \bar{E}[t, t + w] > \theta_2 \wedge \Delta\phi \geq \phi_{min}.$$

# 14 Layer C9: Fixpoint Calibration, Attractor Migration, Carrier Ladder

## 14.1 FixpointCalibrationShell

```

pub struct FixpointCalibrationShell {
    pub calibration_id: Hash256,
    pub current_config_hash: Hash256,
    pub candidate_config_hash: Hash256,
    pub performance_triplet: PerformanceTriplet,
    pub contraction_region_hash: Hash256,
    pub por_acceptance: bool,
    pub resonance_impulse: Option<ResonanceImpulse>,
}

pub struct PerformanceTriplet {
    pub quality: Fixed,
    pub stability: Fixed,
    pub efficiency: Fixed,
}

```

## 14.2 CarrierMigrationPlan

```

pub struct CarrierMigrationPlan {
    pub plan_id: Hash256,
}

```

```

pub source_carrier: Hash256,
pub target_carrier: Hash256,
pub friction_score: Fixed,
pub shock_score: Fixed,
pub anti_drift_score: Fixed,
pub migration_allowed: bool,
pub evidence_refs: Vec<EvidenceRef>,
}

```

### 14.3 PoR-Akzeptanz fuer Kalibrierung

Eine Konfigurationsmutation ist zulaessig, wenn:

$$Q' \geq Q \wedge S' \geq S - \epsilon_S \wedge E' \geq E_{min} \wedge PoR(C') = 1.$$

## 15 Layer C10: Singularity Trigger and Projection Handoff

### 15.1 CognitionCandidateBundle

```

pub struct CognitionCandidateBundle {
    pub bundle_id: Hash256,
    pub cognition_state_id: Hash256,
    pub panorama_id: Hash256,
    pub attractor_map_id: Hash256,
    pub candidate_states: Vec<CognitionCandidate>,
    pub recommended_projection_policy: ProjectionHandoffPolicy,
    pub evidence_refs: Vec<EvidenceRef>,
}

pub struct CognitionCandidate {
    pub candidate_id: Hash256,
    pub source_path_id: Hash256,
    pub target_state_hash: Hash256,
    pub attractor_score: Fixed,
    pub constraint_support_score: Fixed,
    pub memory_support_score: Fixed,
    pub projection_readiness: Fixed,
}

```

### 15.2 ProjectionHandoffPolicy

```

pub enum ProjectionHandoffPolicy {
    Hold,
    ProjectBestCandidate,
    ProjectTopK { k: u32 },
    WaitForHorizon,
    RequestCarrierMigration,
    RequireHumanReview,
}

```

### 15.3 Handoff-Gate

$$G_{handoff} = 1 \iff G_{perc} = 1 \wedge G_{panorama} = 1 \wedge G_{self} = 1 \wedge G_{trigger} = 1 \wedge ReplayReady = 1.$$

## 16 Pipeline

### 16.1 Referenzablauf

1. Lade `CognitionRunDescriptor`, `ProblemSpec`, `TraversalReport` und optionale `Projection-v0.2-Reports`.
2. Canonicalize Inputs; berechne `NullCenter` und `Source-Hashes`.
3. Bilde `CognitiveState5D` aus `Traversal`-, `Signature`-, `Dynamics`- und `Projection`-Metriken.
4. Simuliere oder approximiert aktualisiere den 5D-Zustand mit deterministischer Operator-Algebra.
5. Frage `SpiralMemory` ueber `ResonanceAddressing` ab.
6. Erzeuge `ConstraintLatticeCognition` und `HypercubePuzzleState`.
7. Berechne `Constraint-Perkolation` und optionales `EntropyCollapseCertificate`.
8. Aktualisiere `ScorpioPhaseScheduler` und `ActivationWindows`.
9. Konstruiere `PhasePanorama` mit `Current`- und `Counterfactual-Horizon`.
10. Pruefe und admitiere governierte `Wormholes`.
11. Aktualisiere `SelfModelTensor` und `DualTriggerFeedbackGate`.
12. Fuehre `FixpointCalibration` und `CarrierMigration-Bewertung` aus.
13. Erzeuge `AttractorMap` und ranke Kandidaten.
14. Evaluiere `SingularityTrigger` und `Handoff-Gate`.
15. Erzeuge `CognitionCandidateBundle` oder `CognitionHoldReport`.
16. Schreibe `CognitionReport`, `Audit Trail` und `Replay-Material`.

### 16.2 Pseudocode

```

fn run_cognition(input: CognitionInput, rd: CognitionRunDescriptor)
  -> CognitionOutcome
{
  let canonical = canonicalize(input, rd.canonicalization_version);
  let null_center = derive_null_center(&canonical, &rd);

  let state5 = build_cognitive_state_5d(&canonical, &rd);
  let spiral_hits = spiral_memory_query(&state5, &rd);
  let lattice = build_constraint_lattice(&canonical, &state5, &spiral_hits, &rd);
  let puzzle = build_hypercube_puzzle(&canonical, &lattice, &rd);
  let percolation = evaluate_constraint_percolation(&lattice, &puzzle, &rd);

  let scheduler = scorpio_phase_step(&rd, canonical.logical_step());
  let panorama = build_phase_panorama(
    &state5, &lattice, &puzzle, &scheduler, &spiral_hits, &rd
  );

  let wormholes = govern_wormholes(&panorama, &lattice, &rd);
  let self_model = update_self_model(&state5, &panorama, &wormholes, &rd);
  let feedback = dual_trigger_feedback(&self_model, &rd);
  let calibration = fixpoint_calibrate(&state5, &self_model, &rd);
  let migration = evaluate_carrier_migration(&scheduler, &panorama, &rd);

  let attractors = rank_attractors(
    &state5, &panorama, &lattice, &spiral_hits, &calibration, &rd
  );
  let trigger = singularity_trigger(&state5, &attractors, &feedback, &rd);
  let gate = cognition_handoff_gate(
    &percolation, &panorama, &self_model, &trigger, &rd
  );
}

```



```

    if gate.passed {
        CognitionOutcome::CandidateBundle(make_candidate_bundle(
            state5, panorama, attractors, gate, rd
        ))
    } else {
        CognitionOutcome::Hold(make_hold_report(
            state5, panorama, percolation, self_model, trigger, gate, rd
        ))
    }
}

```

## 17 Reports

### 17.1 CognitionReport

```

pub struct CognitionReport {
    pub report_id: Hash256,
    pub rd_hash: Hash256,
    pub cognition_state_id: Hash256,
    pub cognitive_state_5d_hash: Hash256,
    pub spiral_hitset_hash: Hash256,
    pub lattice_hash: Hash256,
    pub hypercube_puzzle_hash: Hash256,
    pub phase_panorama_hash: Hash256,
    pub self_model_tensor_hash: Hash256,
    pub attractor_map_hash: Hash256,
    pub handoff_gate_hash: Hash256,
    pub outcome_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

### 17.2 CognitionHoldReport

```

pub struct CognitionHoldReport {
    pub hold_id: Hash256,
    pub rd_hash: Hash256,
    pub cognition_state_id: Hash256,
    pub failed_gate: CognitionGateKind,
    pub failure_policy: CognitionFailurePolicy,
    pub diagnostics: Vec<CognitionDiagnostic>,
    pub recommended_next_step: Option<CognitionRecoveryAction>,
}

```

```

pub enum CognitionFailurePolicy {
    Hold,
    RefineConstraints,
    ExpandPanorama,
    QuerySpiralMemory,
    SplitHypercube,
    AdmitWormhole,
    WaitForPhaseWindow,
    MigrateCarrier,
    CalibrateOperators,
    RequireHumanReview,
    Abort,
}

```

```
}
```

## 18 CLI-Schnittstelle

```
pse-traverse-cognition inspect <input.json>
pse-traverse-cognition observe <run.json> --rd rd.json --out cognition.json
pse-traverse-cognition state5 <run.json> --rd rd.json --out state5.json
pse-traverse-cognition memory-query <query.json> --rd rd.json --out hits.json
pse-traverse-cognition lattice <state.json> --rd rd.json --out lattice.json
pse-traverse-cognition puzzle <state.json> --rd rd.json --out puzzle.json
pse-traverse-cognition panorama <state.json> --rd rd.json --out panorama.json
pse-traverse-cognition calibrate <benchmark.json> --rd rd.json --out calibration.json
pse-traverse-cognition trigger <panorama.json> --rd rd.json --out trigger.json
pse-traverse-cognition bundle <cognition.json> --out candidates.json
pse-traverse-cognition replay <report-or-bundle.json>
pse-traverse-cognition verify <report-or-certificate.json>
```

## 19 Determinismus, Canonicalization und Audit

### 19.1 Audit-Pfad

Der Audit-Pfad umfasst:

- CognitionRunDescriptor;
- canonical input bytes;
- NullCenter und Source-Hashes;
- CognitiveState5D;
- OperatorSequence und IntegratorSpec;
- SpiralMemoryHitSet;
- ConstraintLatticeCognition;
- HypercubePuzzleState und CollapseCertificate;
- ScorpioPhaseSchedulerState;
- PhasePanorama;
- WormholeAdmissions;
- SelfModelTensor;
- FixpointCalibrationReport;
- CarrierMigrationPlan;
- SingularityTriggerReport;
- CognitionCandidateBundle oder HoldReport.

### 19.2 Verbotene Audit-Eingaben

- Wall-clock timestamps;
- Plattform-Floats;
- ungeordnete HashMap-Iteration;
- unstabile RNG-Quellen;
- OS-spezifische Pfadnormalisierung;
- Netzwerkzustand;
- versteckte Datenkanäle oder steganographische Transformationen.

## 20 Integration in Repository

### 20.1 Modulzuschnitt

```
crates/pse-traverse/src/cognition/
  mod.rs
  primitives.rs
  run_descriptor.rs
  canonical_state.rs
  cognitive_state_5d.rs
  operator_algebra.rs
  simulation.rs
  spiral_memory.rs
  resonance_addressing.rs
  constraint_lattice.rs
  hypercube_puzzle.rs
  phase_panorama.rs
  choice_geometry.rs
  scorpio_phase_scheduler.rs
  wormhole_hypernet.rs
  self_model_tensor.rs
  dual_trigger_feedback.rs
  fixpoint_calibration.rs
  carrier_migration.rs
  singularity_trigger.rs
  attractor_map.rs
  handoff.rs
  report.rs
  replay.rs

tools/pse-traverse-cognition-cli/src/main.rs
```

### 20.2 Feature Flags

| Feature                      | Bedeutung                                        |
|------------------------------|--------------------------------------------------|
| cognition                    | aktiviert Kernzustand, Reports und Handoff.      |
| cognition-cli                | baut CLI.                                        |
| cognition-simulation         | aktiviert 5D-Simulations- und Stabilitätsprobes. |
| cognition-spiral-memory      | aktiviert SpiralMemory und ResonanceAddressing.  |
| cognition-hypercube          | aktiviert ConstraintLattice und HypercubePuzzle. |
| cognition-scorpio-phase      | aktiviert phase scheduler und ActivationWindows. |
| cognition-wormholes          | aktiviert governierte Nichtlokalpfade.           |
| cognition-calibration        | aktiviert FixpointCalibration.                   |
| cognition-projection-handoff | aktiviert Uebergabe an Projection-v0.2.          |

## 21 Testplan

### 21.1 Unit Tests

1. canonical\_cognition\_state\_is\_content\_addressed
2. cognitive\_state\_5d\_normalizes\_fixed\_values
3. potential\_gradient\_proxy\_is\_deterministic
4. singularity\_detector\_opens\_on\_degenerate\_spectrum

5. operator\_algebra\_rejects\_type\_mismatch
6. spiral\_memory\_hitset\_sorts\_by\_resonance\_then\_hash
7. spiral\_resonance\_distance\_is\_deterministic
8. constraint\_lattice\_percolation\_requires\_all\_terms
9. hypercube\_hidden\_single\_detector\_is\_stable
10. entropy\_collapse\_certificate\_changes\_on\_boundary\_change
11. phase\_panorama\_contains\_current\_and\_counterfactual\_horizons
12. scorpio\_phase\_scheduler\_replays\_byte\_identical
13. wormhole\_admission\_requires\_reason\_budget\_ttl
14. self\_model\_tensor\_rejects\_excessive\_drift
15. dual\_trigger\_requires\_ordered\_trigger\_sequence
16. fixpoint\_calibration\_rejects\_quality\_regression
17. carrier\_migration\_requires\_antidrift
18. handoff\_gate\_fails\_closed

## 21.2 Integration Tests

1. cli\_observe\_writes\_cognition\_report
2. cli\_state5\_writes\_fixed\_point\_observables
3. cli\_memory\_query\_returns\_sorted\_hitset
4. cli\_lattice\_writes\_percolation\_report
5. cli\_puzzle\_writes\_entropy\_collapse\_certificate
6. cli\_panorama\_writes\_720\_horizon
7. cli\_trigger\_writes\_singularity\_trigger\_report
8. cli\_bundle\_writes\_projection\_handoff\_bundle
9. cli\_replay\_byte\_identical
10. projection\_v2\_accepts\_cognition\_candidate\_bundle

## 21.3 Property Tests

1. Permutierte Constraint-Eingaben erzeugen identische Lattice-IDs.
2. Gate-Pass bleibt fail-closed bei Verschärfung einzelner Schwellen.
3. SpiralMemory-Hits sind stabil gegen nichtsemantische Reihenfolgeänderungen.
4. Wormhole-Admissibility ist Funktion von Budget, TTL, ReasonCode und Trace.
5. CandidateBundle-ID ist Funktion aller referenzierten Hashes.
6. Zwei identische Runs erzeugen byte-identische Reports.

## 22 Definition of Done

Eine Implementierung ist abgeschlossen, wenn:

1. cargo test -workspace besteht.
2. cargo clippy -workspace - -D warnings besteht.
3. cargo doc -workspace -no-deps ohne Warnings besteht.
4. alle öffentlichen Typen dokumentiert sind.
5. der Kern ohne Netzwerkabhängigkeit kompiliert.
6. kein gate-relevanter Pfad Plattform-Floats nutzt.
7. alle Reports content-addressed sind.
8. Replay fuer CognitionReport, HitSet, Lattice, Puzzle, Panorama und Bundle byte-identisch ist.

9. Cognition Kernel keine `SemanticCrystal` und keine `FinalizedEmission` erzeugt.
10. Projection-v0.2-Handoff nur `CandidateBundles` uebergibt.
11. alle Gates fail-closed sind.
12. alle `RecoveryActions` maschinenlesbar sind.
13. Testplan vollstaendig implementiert ist.

## 23 Coding-Agent-Auftrag

PATCH-ID: PSE-TRAVERSE-COGNITION-01  
 TITLE: Panoptic Phase Cognition Kernel

### GOAL:

Implementiere einen deterministischen, replayfaehigen Kognitions-kern fuer pse-traverse. Der Kernel erzeugt eine kanonische Loesbarkeitskarte aus 5D Cognitive State, SpiralMemory, ConstraintLattice, HypercubePuzzle, ScorpioPhaseScheduler, PhasePanorama, WormholeHypernet, SelfModelTensor, FixpointCalibration, SingularityTrigger und Projection-v0.2-Handoff.

### MUST:

- crates/pse-traverse/src/cognition/ mit den spezifizierten Modulen anlegen;
- alle Kernobjekte und Reports content-addressed implementieren;
- Fixed/CanonicalNumber fuer alle Audit- und Gate-Werte verwenden;
- SpiralMemory-Hitsets deterministisch sortieren;
- Constraint-Percolation und EntropyCollapseCertificate implementieren;
- PhasePanorama mit Current- und Counterfactual-Horizon implementieren;
- ScorpioPhaseScheduler ohne Steganographie/Hidden Channels implementieren;
- Wormholes nur mit ReasonCode, TTL, Budget und AuditTrace zulassen;
- SelfModelTensor und DualTriggerFeedback fail-closed implementieren;
- CandidateBundle an projection-v0.2 uebergabefaeigig machen;
- CLI mit inspect, observe, state5, memory-query, lattice, puzzle, panorama, calibrate, trigger, bundle, replay, verify bereitstellen;
- Tests gemaess Testplan liefern.

### MUST NOT:

- SemanticCrystal oder FinalizedEmission erzeugen;
- echte Randomness, Wall-clock oder Plattform-Floats in Digests schreiben;
- Steganographie, Tarnkanaele oder versteckte Kommunikation implementieren;
- Zeta-, Dollar-, Mass-Control-, Quantum-Hardware-, Kosmologie- oder Bewusstseinsclaims in den Core einbauen;
- bei fehlgeschlagenem Gate CandidateBundles erzeugen.

### SHOULD:

- BTreeMap/BTreeSet fuer kanonische Strukturen verwenden;
- Golden Fixtures fuer observe/panorama/bundle/replay liefern;
- Feature Flags fuer simulation, spiral-memory, hypercube, scorpio-phase, wormholes, calibration und projection-handoff anlegen.

## 24 Referenz-Zustandsmaschine

| Von            | Nach           | Bedingung                               |
|----------------|----------------|-----------------------------------------|
| Idle           | Loading        | Input und RD vorhanden.                 |
| Loading        | Canonicalizing | Schemata gueltig.                       |
| Canonicalizing | State5D        | canonical bytes und NullCenter erzeugt. |

| Von                | Nach               | Bedingung                                                  |
|--------------------|--------------------|------------------------------------------------------------|
| State5D            | SpiralQuery        | 5D-Zustand normalisiert.                                   |
| SpiralQuery        | LatticeBuild       | HitSet erzeugt.                                            |
| LatticeBuild       | PuzzleBuild        | Constraints geordnet und hashbar.                          |
| PuzzleBuild        | Percolation        | CandidateSets und BoundaryContracts erzeugt.               |
| Percolation        | Scheduling         | Constraint-Perkolationsbestanden oder Hold diagnostiziert. |
| Scheduling         | PanoramaBuild      | PhaseSchedulerState erzeugt.                               |
| PanoramaBuild      | WormholeGovernance | Current/Counterfactual Horizon vorhanden.                  |
| WormholeGovernance | SelfModelUpdate    | Wormholes admitted oder verworfen.                         |
| SelfModelUpdate    | Calibration        | SelfModel kohärent.                                        |
| Calibration        | Triggering         | Kalibrierung akzeptiert oder Hold.                         |
| Triggering         | HandoffGate        | SingularityTrigger bewertet.                               |
| HandoffGate        | CandidateBundle    | Gate bestanden.                                            |
| HandoffGate        | Holding            | Gate fehlgeschlagen.                                       |
| CandidateBundle    | Reported           | CognitionReport erzeugt.                                   |
| Holding            | Reported           | HoldReport erzeugt.                                        |
| Reported           | Replayed           | byte-identische Wiederholung erfolgreich.                  |
| jede Phase         | Failed             | Schema-, Hash-, Determinismus- oder Invarianzverletzung.   |

## 25 Schlussformel

$$\text{Cognition}(s, RD) = \text{CandidateBundle}(s)$$

nur wenn  $G_{5D} = 1 \wedge G_{\text{spiral}} = 1 \wedge G_{\text{perc}} = 1 \wedge G_{\text{panorama}} = 1 \wedge G_{\text{self}} = 1 \wedge G_{\text{trigger}} = 1 \wedge \text{ReplayReady} = 1$ .

Andernfalls:

$$\text{Cognition}(s, RD) = \text{Hold}(s, \text{diagnostics}, \text{recovery}).$$